```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class LibraryMonitoringSystem extends JFrame {

    // Main colors
    Color darkBlue = new Color(30, 58, 95);
    Color blue = new Color(41, 128, 185);
    Color lightBlue = new Color(236, 244, 252);
    Color white = Color.WHITE;
    Color darkText = new Color(40, 40, 40);

    JPanel contentPanel;
    CardLayout cardLayout;

    public LibraryMonitoringSystem() {

        setTitle("Library Monitoring System");
        setSize(1200, 700);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        // Main layout
        setLayout(new BorderLayout());

        // Sidebar
```

```
    JPanel sidebar = createSidebar();
    add(sidebar, BorderLayout.WEST);

    // Main content
    cardLayout = new CardLayout();
    contentPanel = new JPanel(cardLayout);

    contentPanel.add(createDashboard(), "Dashboard");
    contentPanel.add(createStudentsPanel(), "Students");
    contentPanel.add(createBooksPanel(), "Books");
    contentPanel.add(createIssuePanel(), "Issue");
    contentPanel.add(createReturnPanel(), "Return");
    contentPanel.add(createFinePanel(), "Fine");
    contentPanel.add(createReportsPanel(), "Reports");

    add(contentPanel, BorderLayout.CENTER);

    // Show dashboard first
    cardLayout.show(contentPanel, "Dashboard");
}

// ================= SIDEBAR =================

private JPanel createSidebar() {

    JPanel sidebar = new JPanel();
    sidebar.setPreferredSize(new Dimension(230, 700));
    sidebar.setBackground(darkBlue);
```

```
sidebar.setLayout(new BorderLayout());

// Logo / Title
JPanel titlePanel = new JPanel();
titlePanel.setBackground(darkBlue);
titlePanel.setLayout(new BoxLayout(titlePanel, BoxLayout.Y_AXIS));
titlePanel.setBorder(new EmptyBorder(25, 15, 25, 15));

JLabel libraryIcon = new JLabel(" 📚 ");
libraryIcon.setFont(new Font("Arial", Font.PLAIN, 35));
libraryIcon.setAlignmentX(Component.CENTER_ALIGNMENT);

JLabel title = new JLabel("LIBRARY");
title.setForeground(Color.WHITE);
title.setFont(new Font("Arial", Font.BOLD, 22));
title.setAlignmentX(Component.CENTER_ALIGNMENT);

JLabel subtitle = new JLabel("MONITORING SYSTEM");
subtitle.setForeground(new Color(190, 210, 230));
subtitle.setFont(new Font("Arial", Font.BOLD, 11));
subtitle.setAlignmentX(Component.CENTER_ALIGNMENT);

titlePanel.add(libraryIcon);
titlePanel.add(title);
titlePanel.add(subtitle);

sidebar.add(titlePanel, BorderLayout.NORTH);
```

```
// Menu
JPanel menuPanel = new JPanel();
menuPanel.setBackground(darkBlue);
menuPanel.setLayout(new GridLayout(8, 1, 0, 8));
menuPanel.setBorder(new EmptyBorder(10, 12, 10, 12));

JButton dashboardBtn = createMenuButton("🏠  Dashboard");
JButton studentBtn = createMenuButton("👨‍🎓  Students");
JButton bookBtn = createMenuButton("📚  Books");
JButton issueBtn = createMenuButton("📖  Issue Book");
JButton returnBtn = createMenuButton("↩  Return Book");
JButton fineBtn = createMenuButton("💰  Fine Management");
JButton reportBtn = createMenuButton("📊  Reports");
JButton logoutBtn = createMenuButton("🚪  Logout");

dashboardBtn.addActionListener(e ->
    cardLayout.show(contentPanel, "Dashboard"));

studentBtn.addActionListener(e ->
    cardLayout.show(contentPanel, "Students"));

bookBtn.addActionListener(e ->
    cardLayout.show(contentPanel, "Books"));

issueBtn.addActionListener(e ->
    cardLayout.show(contentPanel, "Issue"));
```

```
returnBtn.addActionListener(e ->
      cardLayout.show(contentPanel, "Return"));

fineBtn.addActionListener(e ->
      cardLayout.show(contentPanel, "Fine"));

reportBtn.addActionListener(e ->
      cardLayout.show(contentPanel, "Reports"));

logoutBtn.addActionListener(e -> {

   int choice = JOptionPane.showConfirmDialog(
         this,
         "Are you sure you want to logout?",
         "Logout",
         JOptionPane.YES_NO_OPTION
   );

   if (choice == JOptionPane.YES_OPTION) {
      System.exit(0);
   }
});

menuPanel.add(dashboardBtn);
menuPanel.add(studentBtn);
menuPanel.add(bookBtn);
menuPanel.add(issueBtn);
menuPanel.add(returnBtn);
```

```
    menuPanel.add(fineBtn);
    menuPanel.add(reportBtn);
    menuPanel.add(logoutBtn);

    sidebar.add(menuPanel, BorderLayout.CENTER);

    return sidebar;
}

private JButton createMenuButton(String text) {

    JButton button = new JButton(text);

    button.setFont(new Font("Arial", Font.BOLD, 14));
    button.setForeground(Color.WHITE);
    button.setBackground(darkBlue);
    button.setFocusPainted(false);
    button.setBorderPainted(false);
    button.setHorizontalAlignment(SwingConstants.LEFT);
    button.setCursor(new Cursor(Cursor.HAND_CURSOR));

    return button;
}

// ================= DASHBOARD =================

private JPanel createDashboard() {
```

```
JPanel panel = new JPanel(new BorderLayout());
panel.setBackground(lightBlue);
panel.setBorder(new EmptyBorder(25, 30, 25, 30));

JLabel heading = new JLabel("Library Dashboard");
heading.setFont(new Font("Arial", Font.BOLD, 28));
heading.setForeground(darkText);

JLabel welcome = new JLabel(
    "Welcome to Library Monitoring System"
);
welcome.setFont(new Font("Arial", Font.PLAIN, 15));

JPanel header = new JPanel(new BorderLayout());
header.setOpaque(false);

header.add(heading, BorderLayout.NORTH);
header.add(welcome, BorderLayout.SOUTH);

panel.add(header, BorderLayout.NORTH);

// Cards
JPanel cards = new JPanel(new GridLayout(2, 2, 20, 20));
cards.setOpaque(false);
cards.setBorder(new EmptyBorder(30, 0, 30, 0));

cards.add(createDashboardCard(
    "Total Books",
```

```
            "1250",
            blue
        ));

        cards.add(createDashboardCard(
            "Registered Students",
            "450",
            new Color(39, 174, 96)
        ));

        cards.add(createDashboardCard(
            "Books Issued",
            "125",
            new Color(243, 156, 18)
        ));

        cards.add(createDashboardCard(
            "Overdue Books",
            "18",
            new Color(192, 57, 43)
        ));

        panel.add(cards, BorderLayout.CENTER);

        return panel;
    }

    private JPanel createDashboardCard(
```

```
        String title,
        String value,
        Color color) {

    JPanel card = new JPanel();
    card.setBackground(Color.WHITE);
    card.setLayout(new BorderLayout());
    card.setBorder(BorderFactory.createLineBorder(
            new Color(220, 220, 220)
    ));

    JLabel titleLabel = new JLabel(title);
    titleLabel.setFont(new Font("Arial", Font.BOLD, 16));
    titleLabel.setBorder(new EmptyBorder(20, 20, 5, 20));

    JLabel valueLabel = new JLabel(value);
    valueLabel.setFont(new Font("Arial", Font.BOLD, 35));
    valueLabel.setForeground(color);
    valueLabel.setBorder(new EmptyBorder(5, 20, 20, 20));

    card.add(titleLabel, BorderLayout.NORTH);
    card.add(valueLabel, BorderLayout.CENTER);

    return card;
}

// ================= STUDENTS =================
```

```
private JPanel createStudentsPanel() {

    JPanel panel = createMainPanel("Student Management");

    JPanel form = new JPanel(new GridLayout(2, 4, 10, 10));
    form.setBackground(Color.WHITE);
    form.setBorder(new EmptyBorder(15, 15, 15, 15));

    JTextField idField = new JTextField();
    JTextField nameField = new JTextField();
    JTextField departmentField = new JTextField();
    JTextField phoneField = new JTextField();

    form.add(new JLabel("Student ID"));
    form.add(new JLabel("Student Name"));
    form.add(new JLabel("Department"));
    form.add(new JLabel("Phone Number"));

    form.add(idField);
    form.add(nameField);
    form.add(departmentField);
    form.add(phoneField);

    JButton addButton = new JButton("Add Student");
    addButton.setBackground(blue);
    addButton.setForeground(Color.WHITE);

    String[] columns = {
```

```
        "Student ID",
        "Student Name",
        "Department",
        "Phone Number"
};

DefaultTableModel model =
        new DefaultTableModel(columns, 0);

JTable table = new JTable(model);

addButton.addActionListener(e -> {

    if (idField.getText().isEmpty()
            || nameField.getText().isEmpty()) {

        JOptionPane.showMessageDialog(
                this,
                "Please enter Student ID and Name."
        );

    } else {

        model.addRow(new Object[]{
                idField.getText(),
                nameField.getText(),
                departmentField.getText(),
                phoneField.getText()
```

```
        });

        idField.setText("");
        nameField.setText("");
        departmentField.setText("");
        phoneField.setText("");
      }
    });

    JPanel top = new JPanel(new BorderLayout());
    top.setBackground(Color.WHITE);
    top.add(form, BorderLayout.CENTER);
    top.add(addButton, BorderLayout.SOUTH);

    panel.add(top, BorderLayout.NORTH);
    panel.add(new JScrollPane(table), BorderLayout.CENTER);

    return panel;
}

// ================= BOOKS =================

private JPanel createBooksPanel() {

    JPanel panel = createMainPanel("Book Management");

    JPanel form = new JPanel(new GridLayout(2, 4, 10, 10));
    form.setBackground(Color.WHITE);
```

```
form.setBorder(new EmptyBorder(15, 15, 15, 15));

JTextField bookId = new JTextField();
JTextField bookName = new JTextField();
JTextField author = new JTextField();
JTextField category = new JTextField();

form.add(new JLabel("Book ID"));
form.add(new JLabel("Book Name"));
form.add(new JLabel("Author"));
form.add(new JLabel("Category"));

form.add(bookId);
form.add(bookName);
form.add(author);
form.add(category);

JButton addButton = new JButton("Add Book");
addButton.setBackground(blue);
addButton.setForeground(Color.WHITE);

String[] columns = {
    "Book ID",
    "Book Name",
    "Author",
    "Category",
    "Status"
};
```

```
DefaultTableModel model =
      new DefaultTableModel(columns, 0);

JTable table = new JTable(model);

addButton.addActionListener(e -> {

   if (bookId.getText().isEmpty()
         || bookName.getText().isEmpty()) {

      JOptionPane.showMessageDialog(
            this,
            "Please enter Book ID and Book Name."
      );

   } else {

      model.addRow(new Object[]{
            bookId.getText(),
            bookName.getText(),
            author.getText(),
            category.getText(),
            "Available"
      });

      bookId.setText("");
      bookName.setText("");
```

```
            author.setText("");
            category.setText("");
        }
    });

    JPanel top = new JPanel(new BorderLayout());
    top.setBackground(Color.WHITE);
    top.add(form, BorderLayout.CENTER);
    top.add(addButton, BorderLayout.SOUTH);

    panel.add(top, BorderLayout.NORTH);
    panel.add(new JScrollPane(table), BorderLayout.CENTER);

    return panel;
}

// ================= ISSUE BOOK =================

private JPanel createIssuePanel() {

    JPanel panel = createMainPanel("Issue Book");

    JPanel form = new JPanel(
            new GridLayout(5, 2, 15, 15)
    );

    form.setBackground(Color.WHITE);
    form.setBorder(new EmptyBorder(40, 80, 40, 80));
```

```
JTextField studentId = new JTextField();
JTextField bookId = new JTextField();
JTextField issueDate = new JTextField();
JTextField dueDate = new JTextField();

form.add(new JLabel("Student ID"));
form.add(studentId);

form.add(new JLabel("Book ID"));
form.add(bookId);

form.add(new JLabel("Issue Date"));
form.add(issueDate);

form.add(new JLabel("Due Date"));
form.add(dueDate);

JButton issueButton = new JButton("Issue Book");
issueButton.setBackground(blue);
issueButton.setForeground(Color.WHITE);

form.add(new JLabel(""));
form.add(issueButton);

issueButton.addActionListener(e -> {

   JOptionPane.showMessageDialog(
```

```
            this,
            "Book issued successfully!"
        );
    });

    panel.add(form, BorderLayout.NORTH);

    return panel;
}

// ================= RETURN BOOK =================

private JPanel createReturnPanel() {

    JPanel panel = createMainPanel("Return Book");

    JPanel form = new JPanel(
            new GridLayout(4, 2, 15, 15)
    );

    form.setBackground(Color.WHITE);
    form.setBorder(new EmptyBorder(40, 80, 40, 80));

    JTextField studentId = new JTextField();
    JTextField bookId = new JTextField();
    JTextField returnDate = new JTextField();

    form.add(new JLabel("Student ID"));
```

```
    form.add(studentId);

    form.add(new JLabel("Book ID"));
    form.add(bookId);

    form.add(new JLabel("Return Date"));
    form.add(returnDate);

    JButton returnButton = new JButton("Return Book");
    returnButton.setBackground(new Color(39, 174, 96));
    returnButton.setForeground(Color.WHITE);

    form.add(new JLabel(""));
    form.add(returnButton);

    returnButton.addActionListener(e -> {

        JOptionPane.showMessageDialog(
                this,
                "Book returned successfully!"
        );
    });

    panel.add(form, BorderLayout.NORTH);

    return panel;
}
```

```
// ================= FINE ================

private JPanel createFinePanel() {

    JPanel panel = createMainPanel("Fine Management");

    String[] columns = {
            "Student ID",
            "Book ID",
            "Due Date",
            "Return Date",
            "Fine Amount",
            "Payment Status"
    };

    Object[][] data = {
            {"S001", "B101", "01-10-2026",
                    "05-10-2026", "₹40", "Pending"},
            {"S002", "B205", "02-10-2026",
                    "04-10-2026", "₹20", "Paid"}
    };

    JTable table = new JTable(
            new DefaultTableModel(data, columns)
    );

    panel.add(new JScrollPane(table),
            BorderLayout.CENTER);
```

```
    return panel;
}

// ================= REPORTS =================

private JPanel createReportsPanel() {

    JPanel panel = createMainPanel("Reports");

    JPanel reportPanel = new JPanel(
            new GridLayout(2, 2, 20, 20)
    );

    reportPanel.setBackground(Color.WHITE);
    reportPanel.setBorder(
            new EmptyBorder(50, 80, 50, 80)
    );

    JButton issued = new JButton("Issued Books Report");
    JButton returned = new JButton("Returned Books Report");
    JButton overdue = new JButton("Overdue Books Report");
    JButton students = new JButton("Student Report");

    reportPanel.add(issued);
    reportPanel.add(returned);
    reportPanel.add(overdue);
    reportPanel.add(students);
```

```
    panel.add(reportPanel, BorderLayout.CENTER);

    return panel;
}

// ================= COMMON PANEL =================

private JPanel createMainPanel(String title) {

    JPanel panel = new JPanel(new BorderLayout());
    panel.setBackground(lightBlue);
    panel.setBorder(new EmptyBorder(25, 30, 25, 30));

    JLabel heading = new JLabel(title);
    heading.setFont(new Font("Arial", Font.BOLD, 28));
    heading.setForeground(darkText);

    panel.add(heading, BorderLayout.NORTH);

    return panel;
}

// ================= MAIN =================

public static void main(String[] args) {

    SwingUtilities.invokeLater(() -> {
```

```
            LibraryMonitoringSystem app =
                new LibraryMonitoringSystem();

            app.setVisible(true);
        });
    }
}
package daoimpl;

import dao.*;
import database.DBConnection;
import model.*;

import java.sql.*;
import java.time.LocalDate;
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

public class LibraryDAOImpl implements
        StudentDAO, BookDAO, IssueDAO, ReturnDAO, FineDAO {

    // ============== STUDENT MANAGEMENT ==============

    @Override
    public void addStudent(Student s) {
        String sql = "INSERT INTO students VALUES (?, ?, ?, ?)";
```

```
    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, s.getStudentId());
        ps.setString(2, s.getName());
        ps.setString(3, s.getDepartment());
        ps.setString(4, s.getPhone());

        ps.executeUpdate();

    } catch (SQLException e) {
        throw new RuntimeException("Unable to add student", e);
    }
}

@Override
public void updateStudent(Student s) {
    String sql = "UPDATE students SET student_name=?, " +
                 "department=?, phone=? WHERE student_id=?";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, s.getName());
        ps.setString(2, s.getDepartment());
        ps.setString(3, s.getPhone());
        ps.setString(4, s.getStudentId());
```

```
        if (ps.executeUpdate() == 0) {
            throw new IllegalArgumentException("Student not found");
        }

    } catch (SQLException e) {
        throw new RuntimeException("Unable to update student", e);
    }
}

@Override
public void deleteStudent(String id) {
    String sql = "DELETE FROM students WHERE student_id=?";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, id);

        if (ps.executeUpdate() == 0) {
            throw new IllegalArgumentException("Student not found");
        }

    } catch (SQLException e) {
        throw new RuntimeException(
            "Unable to delete student. Check borrowing records.", e);
    }
}
```

```
@Override
public Student getStudent(String id) {
   String sql = "SELECT * FROM students WHERE student_id=?";

   try (Connection con = DBConnection.getConnection();
       PreparedStatement ps = con.prepareStatement(sql)) {

      ps.setString(1, id);

      try (ResultSet rs = ps.executeQuery()) {
         if (rs.next()) {
            return mapStudent(rs);
         }
      }

   } catch (SQLException e) {
      throw new RuntimeException("Unable to find student", e);
   }

   return null;
}

@Override
public List<Student> getAllStudents() {
   List<Student> list = new ArrayList<>();
   String sql = "SELECT * FROM students ORDER BY student_name";
```

```
    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            list.add(mapStudent(rs));
        }

    } catch (SQLException e) {
        throw new RuntimeException("Unable to load students", e);
    }

    return list;
}

private Student mapStudent(ResultSet rs) throws SQLException {
    return new Student(
        rs.getString("student_id"),
        rs.getString("student_name"),
        rs.getString("department"),
        rs.getString("phone")
    );
}

// ============== BOOK MANAGEMENT ==============

@Override
public void addBook(Book b) {
```

```
    String sql = "INSERT INTO books " +
                 "(book_id, book_name, author, category, status) " +
                 "VALUES (?, ?, ?, ?, 'Available')";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, b.getBookId());
        ps.setString(2, b.getBookName());
        ps.setString(3, b.getAuthor());
        ps.setString(4, b.getCategory());

        ps.executeUpdate();

    } catch (SQLException e) {
        throw new RuntimeException("Unable to add book", e);
    }
}

@Override
public void updateBook(Book b) {
    String sql = "UPDATE books SET book_name=?, author=?, " +
                 "category=? WHERE book_id=?";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, b.getBookName());
```

```
        ps.setString(2, b.getAuthor());
        ps.setString(3, b.getCategory());
        ps.setString(4, b.getBookId());

        if (ps.executeUpdate() == 0) {
            throw new IllegalArgumentException("Book not found");
        }

    } catch (SQLException e) {
        throw new RuntimeException("Unable to update book", e);
    }
}

@Override
public void deleteBook(String id) {
    String sql = "DELETE FROM books WHERE book_id=?";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, id);

        if (ps.executeUpdate() == 0) {
            throw new IllegalArgumentException("Book not found");
        }

    } catch (SQLException e) {
        throw new RuntimeException(
```

```
            "Unable to delete book. Check its issue history.", e);
        }
    }

    @Override
    public Book getBook(String id) {
        String sql = "SELECT * FROM books WHERE book_id=?";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, id);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return mapBook(rs);
                }
            }

        } catch (SQLException e) {
            throw new RuntimeException("Unable to find book", e);
        }

        return null;
    }

    @Override
    public List<Book> getAllBooks() {
```

```
    List<Book> list = new ArrayList<>();
    String sql = "SELECT * FROM books ORDER BY book_name";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            list.add(mapBook(rs));
        }

    } catch (SQLException e) {
        throw new RuntimeException("Unable to load books", e);
    }

    return list;
}

@Override
public List<Book> searchBooks(String keyword) {
    List<Book> list = new ArrayList<>();

    String sql = "SELECT * FROM books WHERE " +
                 "book_name LIKE ? OR author LIKE ? " +
                 "OR category LIKE ? ORDER BY book_name";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {
```

```
            String search = "%" + keyword + "%";
            ps.setString(1, search);
            ps.setString(2, search);
            ps.setString(3, search);

            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    list.add(mapBook(rs));
                }
            }

        } catch (SQLException e) {
            throw new RuntimeException("Unable to search books", e);
        }

        return list;
    }

    private Book mapBook(ResultSet rs) throws SQLException {
        return new Book(
            rs.getString("book_id"),
            rs.getString("book_name"),
            rs.getString("author"),
            rs.getString("category"),
            rs.getString("status")
        );
    }
```

```
// ============== ISSUE BOOK ==============

@Override
public int issueBook(String studentId, String bookId,
                     int loanDays) {

  if (loanDays <= 0) {
    throw new IllegalArgumentException(
      "Loan days must be greater than zero");
  }

  String studentSql =
    "SELECT student_id FROM students WHERE student_id=?";

  String bookSql =
    "SELECT status FROM books WHERE book_id=? FOR UPDATE";

  String issueSql =
    "INSERT INTO issue_books " +
    "(student_id, book_id, issue_date, due_date, status) " +
    "VALUES (?, ?, ?, ?, 'Issued')";

  String updateBookSql =
    "UPDATE books SET status='Issued' " +
    "WHERE book_id=? AND status='Available'";

  try (Connection con = DBConnection.getConnection()) {
```

```
con.setAutoCommit(false);

try {
   // Verify student
   try (PreparedStatement ps =
         con.prepareStatement(studentSql)) {
      ps.setString(1, studentId);

      try (ResultSet rs = ps.executeQuery()) {
         if (!rs.next()) {
            throw new IllegalArgumentException(
               "Student does not exist");
         }
      }
   }

   // Lock and verify book
   try (PreparedStatement ps =
         con.prepareStatement(bookSql)) {
      ps.setString(1, bookId);

      try (ResultSet rs = ps.executeQuery()) {
         if (!rs.next()) {
            throw new IllegalArgumentException(
               "Book does not exist");
         }

         if (!"Available".equals(rs.getString("status"))) {
```

```
            throw new IllegalStateException(
                "Book is not available");
        }
    }
}

LocalDate issueDate = LocalDate.now();
LocalDate dueDate = issueDate.plusDays(loanDays);

// Save issue record
int issueId;

try (PreparedStatement ps = con.prepareStatement(
        issueSql, Statement.RETURN_GENERATED_KEYS)) {

    ps.setString(1, studentId);
    ps.setString(2, bookId);
    ps.setDate(3, Date.valueOf(issueDate));
    ps.setDate(4, Date.valueOf(dueDate));

    ps.executeUpdate();

    try (ResultSet keys = ps.getGeneratedKeys()) {
        if (!keys.next()) {
            throw new SQLException(
                "Issue ID was not generated");
        }
        issueId = keys.getInt(1);
```

```
            }
        }

        // Update book availability
        try (PreparedStatement ps =
                con.prepareStatement(updateBookSql)) {
            ps.setString(1, bookId);

            if (ps.executeUpdate() != 1) {
                throw new IllegalStateException(
                    "Unable to update book status");
            }
        }

        con.commit();
        return issueId;

    } catch (Exception e) {
        con.rollback();
        throw e;
    } finally {
        con.setAutoCommit(true);
    }

  } catch (Exception e) {
    throw new RuntimeException("Book issue failed", e);
  }
}
```

```
@Override
public List<IssueBook> getAllIssues() {
   List<IssueBook> list = new ArrayList<>();

   String sql = "SELECT * FROM issue_books ORDER BY issue_id DESC";

   try (Connection con = DBConnection.getConnection();
       PreparedStatement ps = con.prepareStatement(sql);
       ResultSet rs = ps.executeQuery()) {

      while (rs.next()) {
         list.add(mapIssue(rs));
      }

   } catch (SQLException e) {
      throw new RuntimeException("Unable to load issues", e);
   }

   return list;
}

@Override
public List<IssueBook> getStudentHistory(String studentId) {
   List<IssueBook> list = new ArrayList<>();

   String sql = "SELECT * FROM issue_books " +
            "WHERE student_id=? ORDER BY issue_date DESC";
```

```
    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, studentId);

        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                list.add(mapIssue(rs));
            }
        }

    } catch (SQLException e) {
        throw new RuntimeException(
            "Unable to load borrowing history", e);
    }

    return list;
}

private IssueBook mapIssue(ResultSet rs) throws SQLException {
    return new IssueBook(
        rs.getInt("issue_id"),
        rs.getString("student_id"),
        rs.getString("book_id"),
        rs.getDate("issue_date").toLocalDate(),
        rs.getDate("due_date").toLocalDate(),
        rs.getString("status")
```

```
    );
}

// ============== RETURN BOOK & FINE ==============

@Override
public BigDecimal returnBook(int issueId,
                             LocalDate returnDate) {

    final BigDecimal dailyFine = new BigDecimal("10.00");

    String issueSql =
        "SELECT book_id, due_date, status FROM issue_books " +
        "WHERE issue_id=? FOR UPDATE";

    String returnSql =
        "INSERT INTO return_books (issue_id, return_date) " +
        "VALUES (?, ?)";

    String updateIssueSql =
        "UPDATE issue_books SET status='Returned' " +
        "WHERE issue_id=? AND status='Issued'";

    String updateBookSql =
        "UPDATE books SET status='Available' WHERE book_id=?";

    String fineSql =
        "INSERT INTO fines (issue_id, fine_amount, payment_status) " +
```

```
    "VALUES (?, ?, ?)";

try (Connection con = DBConnection.getConnection()) {
    con.setAutoCommit(false);

    try {
        String bookId;
        LocalDate dueDate;

        try (PreparedStatement ps =
                con.prepareStatement(issueSql)) {
            ps.setInt(1, issueId);

            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) {
                    throw new IllegalArgumentException(
                        "Issue record not found");
                }

                if (!"Issued".equals(rs.getString("status"))) {
                    throw new IllegalStateException(
                        "Book has already been returned");
                }

                bookId = rs.getString("book_id");
                dueDate = rs.getDate("due_date").toLocalDate();
            }
        }
```

```
long overdueDays = Math.max(
   0, java.time.temporal.ChronoUnit.DAYS.between(
      dueDate, returnDate)
);

BigDecimal amount = dailyFine.multiply(
   BigDecimal.valueOf(overdueDays)
);

try (PreparedStatement ps =
       con.prepareStatement(returnSql)) {
   ps.setInt(1, issueId);
   ps.setDate(2, Date.valueOf(returnDate));
   ps.executeUpdate();
}

try (PreparedStatement ps =
       con.prepareStatement(updateIssueSql)) {
   ps.setInt(1, issueId);

   if (ps.executeUpdate() != 1) {
      throw new IllegalStateException(
         "Issue status could not be updated");
   }
}

try (PreparedStatement ps =
```

```
            con.prepareStatement(updateBookSql)) {
        ps.setString(1, bookId);
        ps.executeUpdate();
      }

      String paymentStatus =
        amount.compareTo(BigDecimal.ZERO) == 0
          ? "Paid" : "Pending";

      try (PreparedStatement ps =
            con.prepareStatement(fineSql)) {
        ps.setInt(1, issueId);
        ps.setBigDecimal(2, amount);
        ps.setString(3, paymentStatus);
        ps.executeUpdate();
      }

      con.commit();
      return amount;

    } catch (Exception e) {
      con.rollback();
      throw e;
    } finally {
      con.setAutoCommit(true);
    }

  } catch (Exception e) {
```

```
        throw new RuntimeException("Book return failed", e);
    }
}

// ============== FINE MANAGEMENT ==============

@Override
public List<Fine> getAllFines() {
    List<Fine> list = new ArrayList<>();

    String sql = "SELECT * FROM fines ORDER BY fine_id DESC";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            list.add(new Fine(
                rs.getInt("fine_id"),
                rs.getInt("issue_id"),
                rs.getBigDecimal("fine_amount"),
                rs.getString("payment_status")
            ));
        }

    } catch (SQLException e) {
        throw new RuntimeException("Unable to load fines", e);
    }
```

```
    return list;
}

@Override
public BigDecimal payFine(int fineId, BigDecimal amount) {
    if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException(
            "Payment must be greater than zero");
    }

    try (Connection con = DBConnection.getConnection()) {
        con.setAutoCommit(false);

        try {
            BigDecimal total;
            BigDecimal alreadyPaid;

            String sql = "SELECT fine_amount, payment_status " +
                "FROM fines WHERE fine_id=? FOR UPDATE";

            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, fineId);

                try (ResultSet rs = ps.executeQuery()) {
                    if (!rs.next()) {
                        throw new IllegalArgumentException(
                            "Fine record not found");
```

```
        }

        total = rs.getBigDecimal("fine_amount");

        String status = rs.getString("payment_status");
        if ("Paid".equals(status)) {
           throw new IllegalStateException(
              "Fine is already paid");
        }
     }
  }

  String paidSql =
     "SELECT COALESCE(SUM(amount), 0) AS paid " +
     "FROM fine_payments WHERE fine_id=?";

  try (PreparedStatement ps =
         con.prepareStatement(paidSql)) {
     ps.setInt(1, fineId);

     try (ResultSet rs = ps.executeQuery()) {
        rs.next();
        alreadyPaid = rs.getBigDecimal("paid");
     }
  }
```